



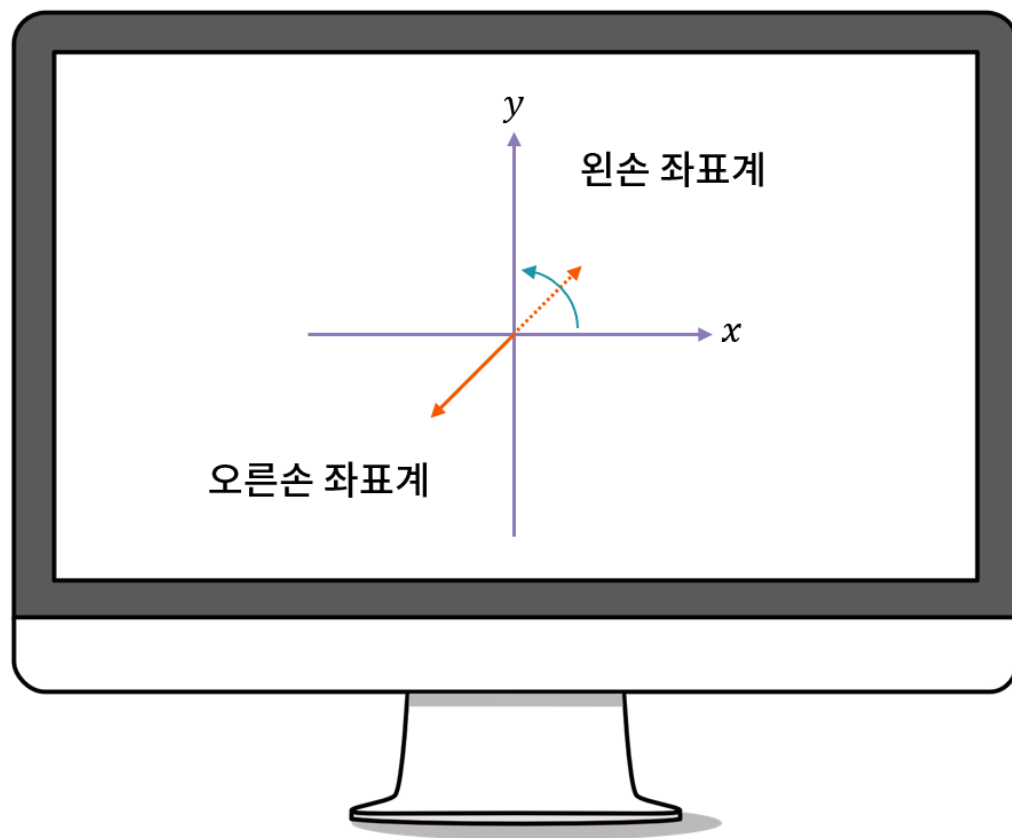
10. 3차원 공간

입체 공간의 생성





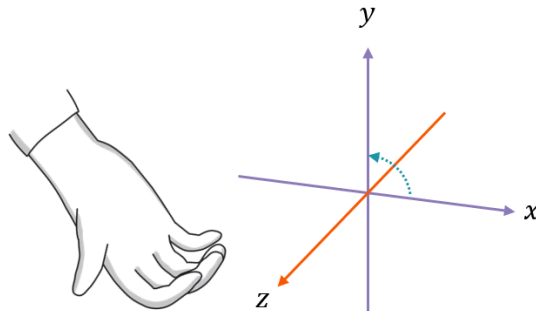
- 오른손 좌표계 vs 왼손 좌표계



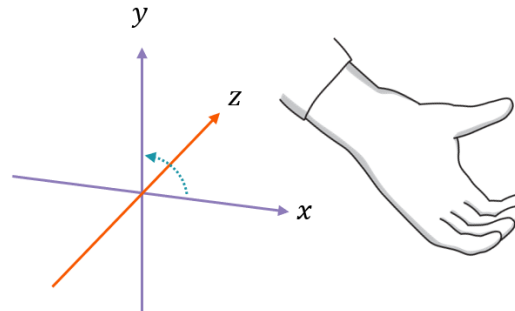


- 오른손 좌표계 왼손 좌표계

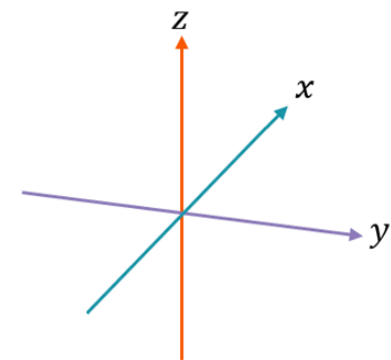
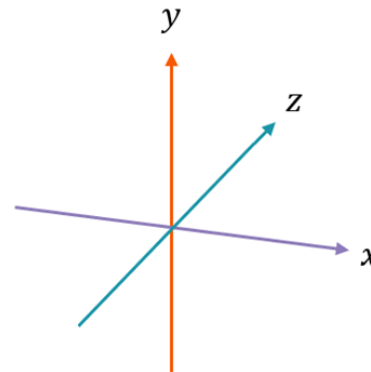
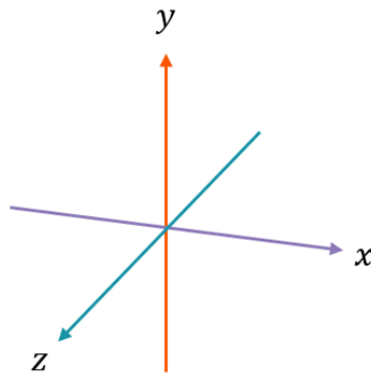
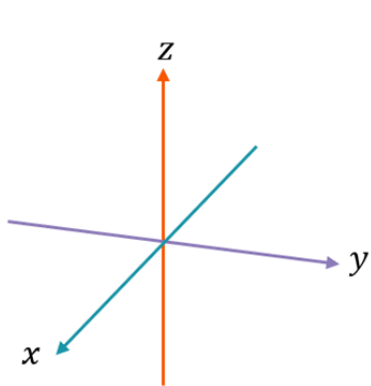
오른손 좌표계



왼손 좌표계



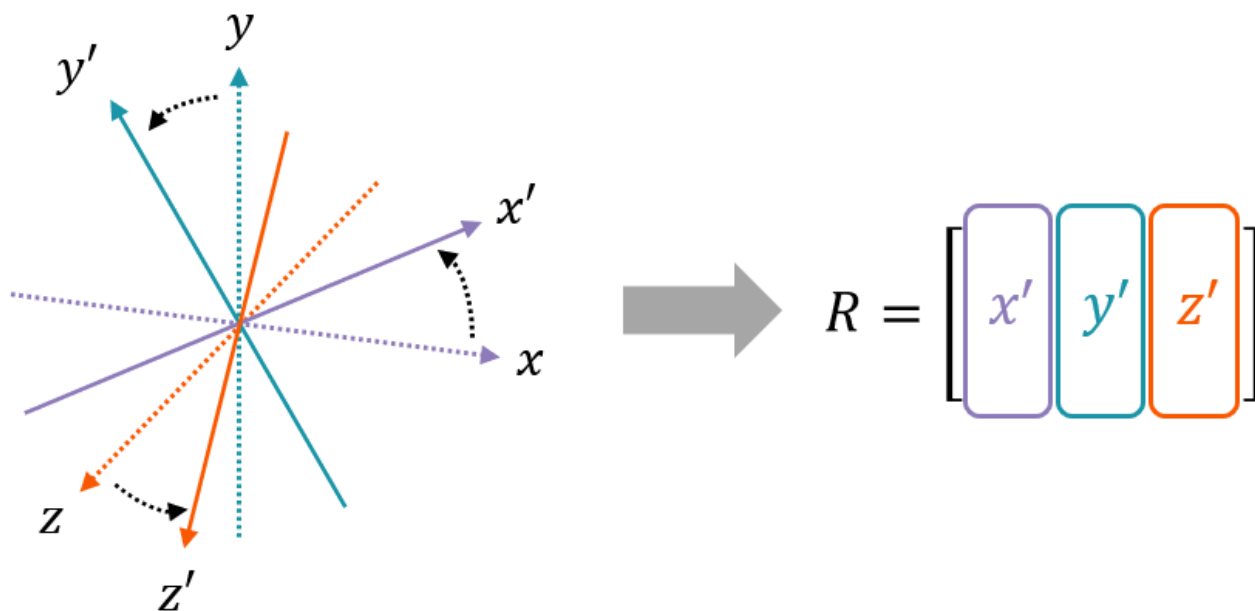
- 3D Max, OpenGL, Unity, Unreal





- 세 축의 기저벡터로 만들어지는 회전행렬

$$S = \begin{bmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad T = \begin{bmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

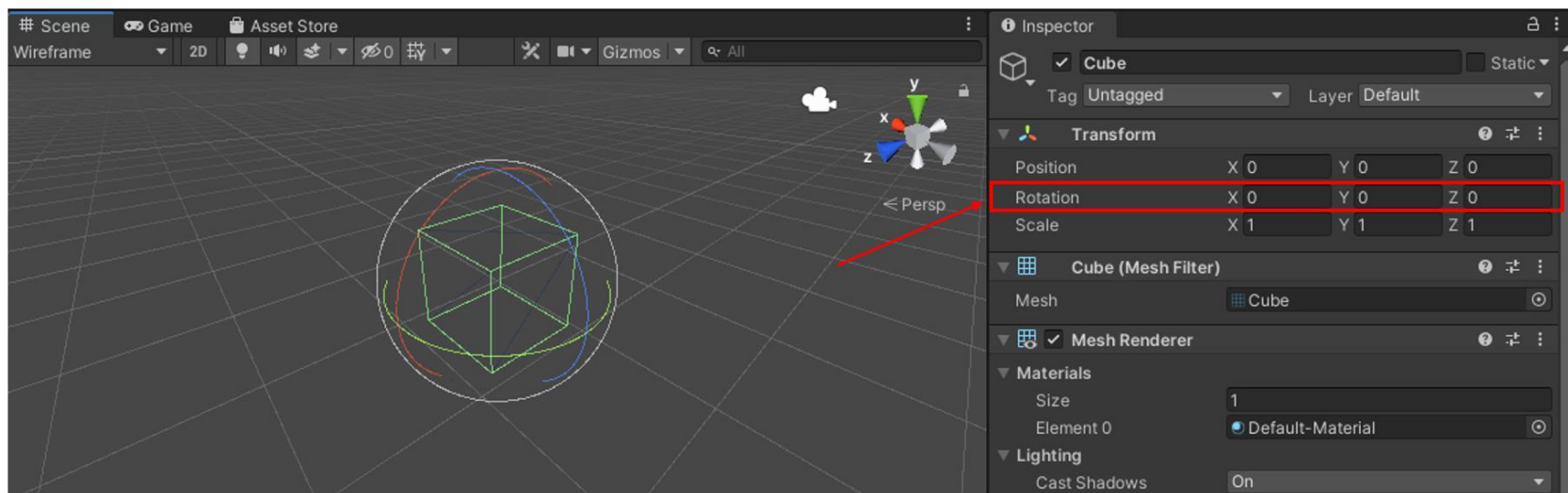




- 오일러 회전

- ◆ 3차원 좌표계(x, y, z)에서 X, Y, Z 축을 기준으로 물체를 순차적으로 회전, 방향을 결정하는 방식

- 유니티 엔진 오일러 각 3차원 회전



$$(\theta_x, \theta_y, \theta_z)$$

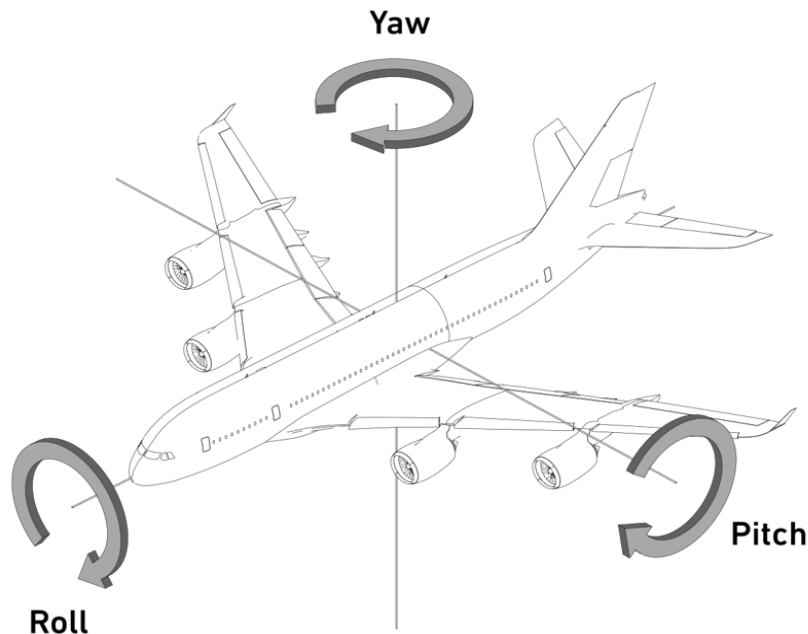




- 오일러 각: vector 표현

$$(\theta_x, \theta_y, \theta_z)$$

- 요(Y: Yaw), 롤(Z: Roll), 피치(X: Pitch) 회전
 - 표준 기저 벡터 축을 기준으로 회전 ← 항공기 회전 표현, 게임에서 사용





- X 축 회전

$$R_x = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \theta & -\sin \theta \\ 0 & \sin \theta & \cos \theta \end{bmatrix}$$

- Y 축 회전

$$R_y = \begin{bmatrix} \cos \theta & 0 & \sin \theta \\ 0 & 1 & 0 \\ -\sin \theta & 0 & \cos \theta \end{bmatrix}$$

- Z 축 회전

$$R_z = \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

- 축 회전: $X \rightarrow Y \rightarrow Z \rightarrow X \rightarrow Y$

- ◆ Y축에 직교하는 평면: $zx(0) \ xz(X)$
- ◆ X축에 직교하는 평면: yz
- ◆ Z축에 직교하는 평면: xy





● 오일러 각 조합에 대한 경우의 수

경우	회전축의 순서	회전 동작의 순서
1	$x \rightarrow y \rightarrow z$	피치 → 요 → 롤
2	$x \rightarrow z \rightarrow y$	피치 → 롤 → 요
3	$y \rightarrow x \rightarrow z$	요 → 피치 → 롤
4	$y \rightarrow z \rightarrow x$	요 → 롤 → 피치
6	$z \rightarrow y \rightarrow x$	롤 → 요 → 피치

● $R = R_{yaw} \cdot R_{pitch} \cdot R_{roll}$

$$R = R_{\alpha} \cdot R_{\beta} \cdot R_{\gamma}$$

$$= \begin{bmatrix} \cos \alpha & 0 & \sin \alpha \\ 0 & 1 & 0 \\ -\sin \alpha & 0 & \cos \alpha \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \beta & -\sin \beta \\ 0 & \sin \beta & \cos \beta \end{bmatrix} \begin{bmatrix} \cos \gamma & -\sin \gamma & 0 \\ \sin \gamma & \cos \gamma & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$= \begin{bmatrix} \cos \alpha \cos \gamma + \sin \alpha \sin \beta \sin \gamma & -\cos \alpha \sin \gamma + \sin \alpha \sin \beta \cos \gamma & \sin \alpha \cos \beta \\ \cos \beta \sin \gamma & \cos \beta \cos \gamma & -\sin \beta \\ -\sin \theta \cos \gamma + \cos \alpha \sin \beta \sin \gamma & \sin \alpha \sin \gamma + \cos \alpha \sin \beta \cos \gamma & \cos \alpha \cos \beta \end{bmatrix}$$





- X, Y, Z

$$X_{local} = (\cos \alpha \cos \gamma + \sin \alpha \sin \beta \sin \gamma, \cos \beta \sin \gamma, -\sin \alpha \cos \gamma + \cos \alpha \sin \beta \sin \gamma)$$

$$Y_{local} = (-\cos \alpha \sin \gamma + \sin \alpha \sin \beta \cos \gamma, \cos \beta \cos \gamma, \sin \alpha \sin \gamma + \cos \alpha \sin \beta \cos \gamma)$$

$$Z_{local} = (\sin \alpha \cos \beta, -\sin \beta, \cos \alpha \cos \beta)$$

- 로컬 축 벡터와 회전 행렬

- ◆ 로컬 축 벡터

$$\vec{x} = (x_x, x_y, x_z) \quad \vec{y} = (y_x, y_y, y_z) \quad \vec{z} = (z_x, z_y, z_z)$$

- ◆ 회전 행렬

$$R_{RHL} = \begin{bmatrix} x_x & y_x & z_x & 0 \\ x_y & y_y & z_y & 0 \\ x_z & y_z & z_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$





- 크기(S), 회전(R), 이동(T)

$$S = \begin{bmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad R_{RHL} = \begin{bmatrix} x_x & y_x & z_x & 0 \\ x_y & y_y & z_y & 0 \\ x_z & y_z & z_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad T = \begin{bmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

- T·R·S 모델링행렬 M

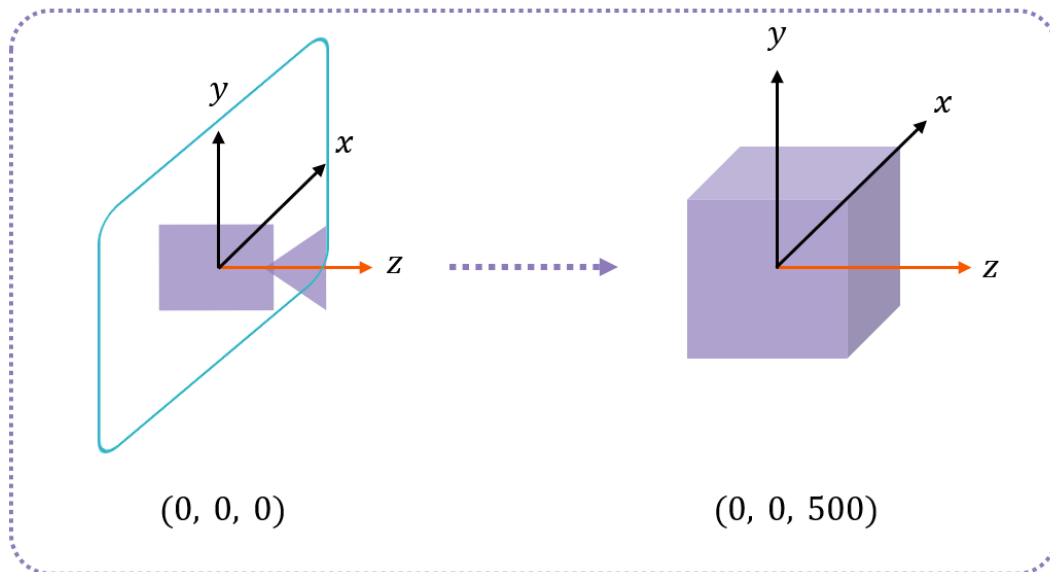
$$M = T \cdot R \cdot S = \begin{bmatrix} x_x s_x & y_x s_y & z_x s_z & t_x \\ x_y s_x & y_y s_y & z_y s_z & t_y \\ x_z s_x & y_z s_y & z_z s_z & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$



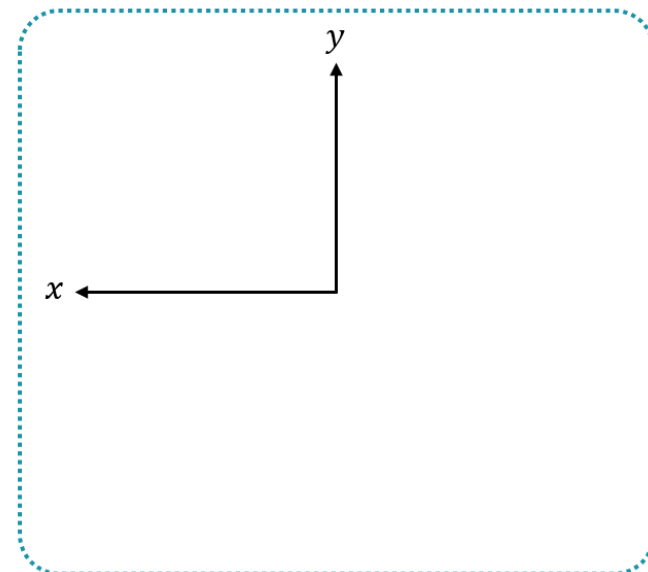


● 3차원 공간의 카메라 설정

월드 공간



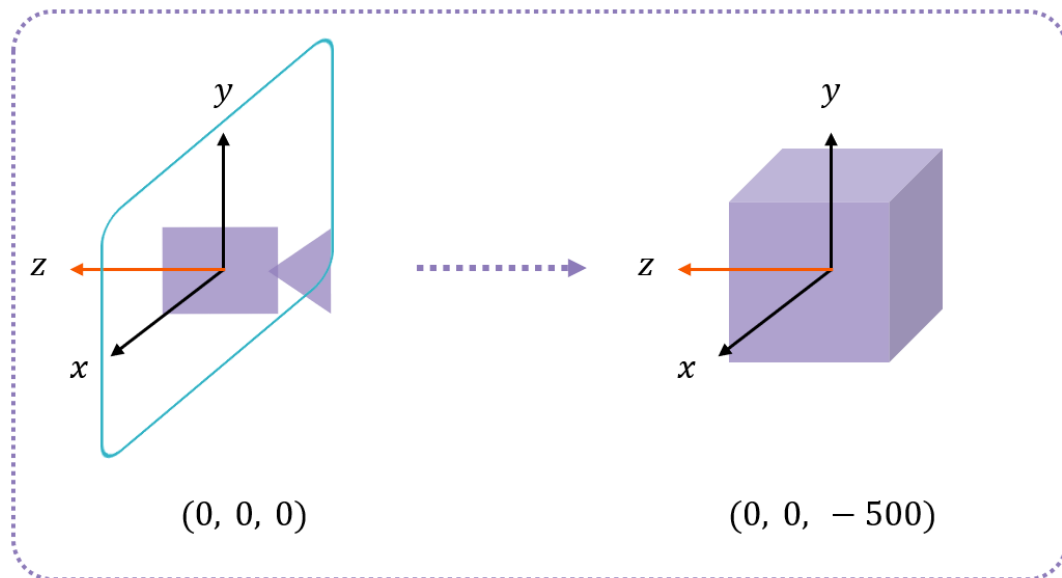
카메라가 만드는 화면 좌표계



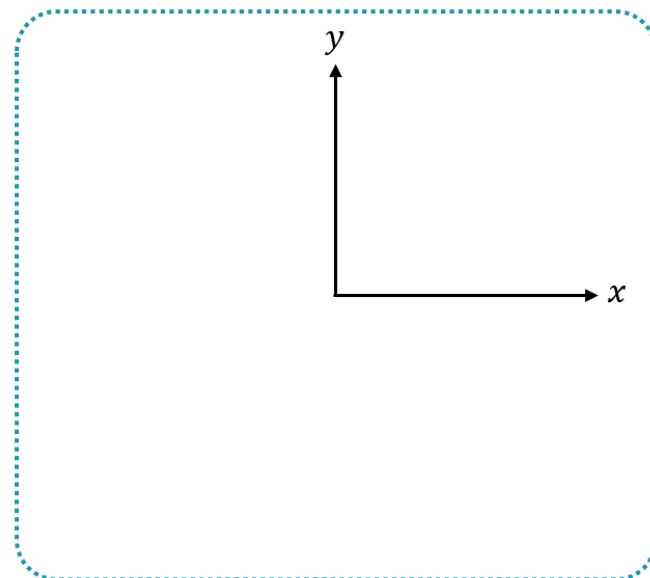


- 뷰 공간으로 재해석된 3차원 공간

뷰 공간

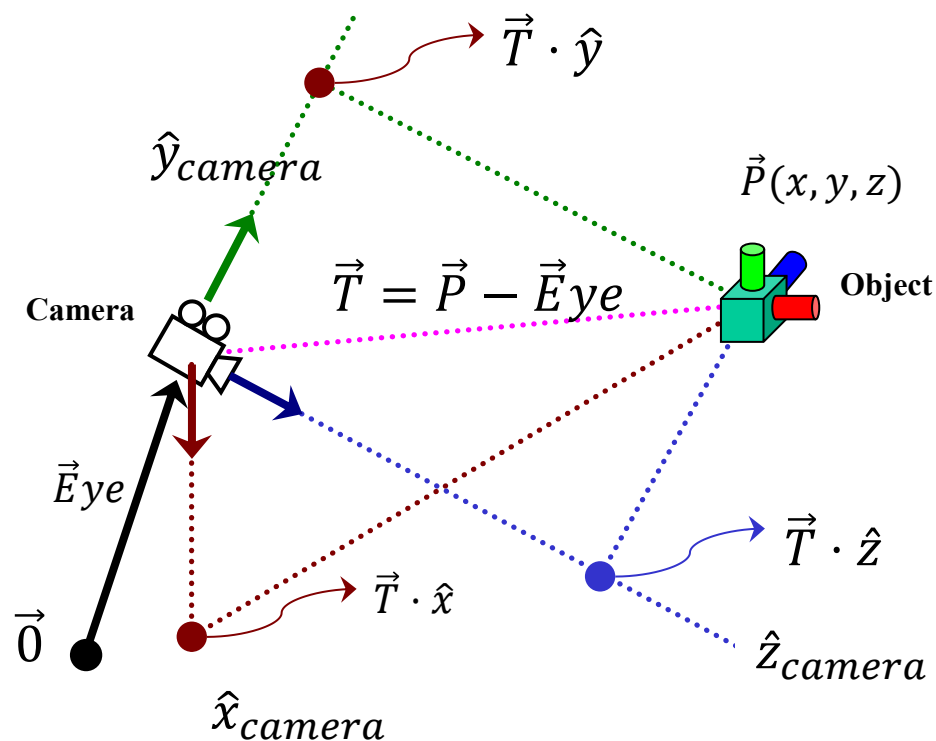


카메라가 만드는 화면 좌표계





- 뷰 공간으로 재해석된 3차원 공간





● 뷰 행렬

◆ 카메라 지역 공간으로 이동

- 카메라 이동의 역행렬 → 회전의 역행렬 적용
- 크기 변환 없음. 회전, 이동 변환 만 존재

◆ 카메라 축: $\hat{x}, \hat{y}, \hat{z}$, 카메라 transform : $\vec{t} = (t_x, t_y, t_z)$

● $M_{\text{view}} = R_{\text{camera}}^{-1} \cdot T_{\text{camera}}^{-1}$

$$R^{-1} = \begin{bmatrix} x_x & x_y & x_z & 0 \\ y_x & y_y & y_z & 0 \\ z_x & z_y & z_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad T = \begin{bmatrix} 1 & 0 & 0 & -t_x \\ 0 & 1 & 0 & -t_y \\ 0 & 0 & 1 & -t_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$M_{\text{view}} = \begin{bmatrix} x_x & x_y & x_z & -\hat{x} \cdot \vec{t} \\ y_x & y_y & y_z & -\hat{y} \cdot \vec{t} \\ z_x & z_y & z_z & -\hat{z} \cdot \vec{t} \\ 0 & 0 & 0 & 1 \end{bmatrix}$$





- Y 축으로 180도 회전

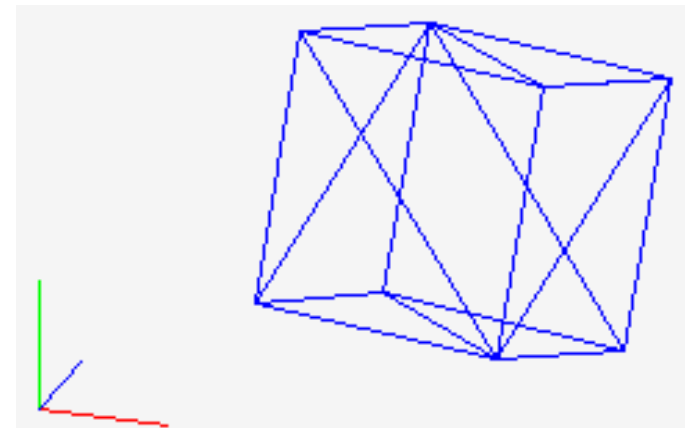
$$V = \begin{bmatrix} -x_x & -x_y & -x_z & \hat{x} \cdot \vec{t} \\ y_x & y_y & y_z & -\hat{y} \cdot \vec{t} \\ -z_x & z_y & -z_z & +\hat{z} \cdot \vec{t} \\ 0 & 0 & 0 & 1 \end{bmatrix}$$





10-1 처음 그려보는 3차원 공간

```
FORCEINLINE void CameraObject::GetViewAxes(Vector3& OutViewX, Vector3& OutViewY, Vector3& OutViewZ) const
{
    // 로컬 Z축 값의 반대 방향을 뷰 공간의 Z축으로 설정
    OutViewZ = -_Transform.GetLocalZ();
    OutViewY = _Transform.GetLocalY();
    OutViewX = -_Transform.GetLocalX();
}
```



```
FORCEINLINE Matrix4x4 CameraObject::GetViewMatrix() const
{
    Vector3 viewX, viewY, viewZ;
    GetViewAxes(viewX, viewY, viewZ);
    Vector3 pos = _Transform.GetPosition();

    return Matrix4x4(
        Vector4(Vector3(viewX.X, viewY.X, viewZ.X), false),
        Vector4(Vector3(viewX.Y, viewY.Y, viewZ.Y), false),
        Vector4(Vector3(viewX.Z, viewY.Z, viewZ.Z), false),
        Vector4(-viewX.Dot(pos), -viewY.Dot(pos), -viewZ.Dot(pos), 1.f)
    );
}
```





- 장점

- ◆ 회전에 대한 직관적인 인터페이스 제공
- ◆ 3개의 데이터 만 있으면 회전을 표현
(회전을 표현하려면 3×3 행렬원소 필요)
- ◆ 직관적인 표준 기저 벡터 를 회전축으로 사용 → 회전 설계 용이
- ◆ 회전 설정에서 가장 적합 → 대다수 3차원 소프트웨어에서 사용

- 단점

- ◆ 짐벌락 현상





- 짐벌락 현상
 - ◆ 특정 각도에서 두 개의 회전축이 겹쳐, 하나의 자유도 상실
- 발생 원인
 - ◆ 행렬 연산 순서의 종속성
- 해결 방법
 - ◆ 쿼터니언(Quaternion) 사용





● 오일러 각도로 인한 자유도 손실

https://en.wikipedia.org/wiki/Gimbal_lock

$$R = R_\alpha \cdot R_\beta \cdot R_\gamma$$

$$= \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \alpha & -\sin \alpha \\ 0 & \sin \alpha & \cos \alpha \end{bmatrix} \begin{bmatrix} \cos \beta & 0 & \sin \beta \\ 0 & 1 & 0 \\ -\sin \beta & 0 & \cos \beta \end{bmatrix} \begin{bmatrix} \cos \gamma & -\sin \gamma & 0 \\ \sin \gamma & \cos \gamma & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

● $\beta = \pi/2$

$$R = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \alpha & -\sin \alpha \\ 0 & \sin \alpha & \cos \alpha \end{bmatrix} \begin{bmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ -1 & 0 & 0 \end{bmatrix} \begin{bmatrix} \cos \gamma & -\sin \gamma & 0 \\ \sin \gamma & \cos \gamma & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$= \begin{bmatrix} 0 & 0 & 1 \\ \sin \alpha \cos \gamma + \cos \alpha \sin \gamma & -\sin \alpha \sin \gamma + \cos \alpha \cos \gamma & 0 \\ -\cos \alpha \cos \gamma + \sin \alpha \sin \gamma & \cos \alpha \sin \gamma + \sin \alpha \cos \gamma & 0 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 1 \\ \sin(\alpha + \gamma) & \cos(\alpha + \gamma) & 0 \\ -\cos(\alpha + \gamma) & \sin(\alpha + \gamma) & 0 \end{bmatrix}$$

$$R = f(\alpha + \gamma) = f((\alpha + \Delta) + \gamma) = f(\alpha + (\gamma + \Delta))$$

◆ α 와 γ 의 회전 효과를 구분할 수 없음

- 수학적: 회전 행렬이 α 와 γ 를 독립 변수로 가지지 않고 $\alpha + \gamma$ 로만 표현
- 물리적: 두 회전 축이 정렬되어 자유도 1개가 소실





```
// 입력에 따른 플레이어 트랜스폼의 변경
Rotator r = goPlayer.GetTransform().GetRotation();
if (input.IsPressed(InputButton::Space))
{
    r.Pitch = -90.f;
}
else
{
    r.Pitch += input.GetAxis(InputAxis::ZAxis) * rotateSpeed * InDeltaSeconds;
}
r.Roll += input.GetAxis(InputAxis::XAxis) * rotateSpeed * InDeltaSeconds;
r.Yaw += input.GetAxis(InputAxis::YAxis) * rotateSpeed * InDeltaSeconds;
```





- 회전의 선형 보간

$$\theta' = (1 - t) \theta_{start} + t \theta_{end}$$

- 특정 축 회전 $R(\alpha, 0, 0)$

$$R(\alpha, \theta, \varphi) = R_\alpha \cdot R_\theta \cdot R_\varphi$$

$$R(\beta, \theta, \varphi) = R_\beta \cdot R_\theta \cdot R_\varphi$$

$$R(\beta, \theta, \varphi) \cdot R(\alpha, \theta, \varphi) = R_\beta \cdot R_\theta \cdot R_\varphi \cdot R_\alpha \cdot R_\theta \cdot R_\varphi = R_\beta \cdot I \cdot I \cdot R_\alpha \cdot I \cdot I$$

$$R(\beta, \theta, \varphi) \cdot R(\alpha, \theta, \varphi) = R_\beta \cdot R_\alpha = R_{(\alpha+\beta)}$$

- 특정 축 회전 $R(\alpha, \alpha, 0)$

$$R(\beta) \cdot R(\alpha) \neq R_{(\alpha+\beta)}$$





- 3차원 트랜스폼

- ◆ T·R·S

- 카메라 뷰 행렬

$$M_{view} = \begin{bmatrix} x_x & x_y & x_z & -\hat{x} \cdot \vec{t} \\ y_x & y_y & y_z & -\hat{y} \cdot \vec{t} \\ z_x & z_y & z_z & -\hat{z} \cdot \vec{t} \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

- 오일러 각

- ◆ 짐벌락

- ◆ 사원수 보간





11. 외적

3차원 공간의 분석과 응용





- 외적
 - ◆ Cross product, Outer product
- 다양한 외적 계산 방법

$$\vec{u} = (u_x, u_y, u_z) \quad \vec{v} = (v_x, v_y, v_z)$$

$$\vec{u} \times \vec{v} = |\vec{u}| |\vec{v}| \sin \theta \hat{n}$$

$$\vec{u} \times \vec{v} = \varepsilon_{ijk} u_i v_j \hat{e}_k$$

$$\vec{u} \times \vec{v} = \begin{vmatrix} \hat{x} & \hat{y} & \hat{z} \\ u_x & u_y & u_z \\ v_x & v_y & v_z \end{vmatrix}$$

$$\vec{u} \times \vec{v} = (u_y v_z - u_z v_y, u_z v_x - u_x v_z, u_x v_y - u_y v_x)$$





• $\vec{u} \times \vec{v} = |\vec{u}| |\vec{v}| \sin \theta \hat{n}$ 전개

$$|\vec{u} \times \vec{v}|^2 = |\vec{u}|^2 |\vec{v}|^2 \sin^2 \theta$$

$$= |\vec{u}|^2 |\vec{v}|^2 (1 - \cos^2 \theta)$$

$$= |\vec{u}|^2 |\vec{v}|^2 - |\vec{u}|^2 |\vec{v}|^2 \cos^2 \theta$$

$$= |\vec{u}|^2 |\vec{v}|^2 - (\vec{u} \cdot \vec{v})^2$$

$$= (u_x u_x + u_y u_y + u_z u_z)(v_x v_x + v_y v_y + v_z v_z) - (u_x v_x + u_y v_y + u_z v_z)^2$$

$$= u_x u_x v_x v_x + u_y u_y v_x v_x + u_z u_z v_x v_x$$

$$+ u_x u_x v_y v_y + u_y u_y v_y v_y + u_z u_z v_y v_y$$

$$+ u_x u_x v_z v_z + u_y u_y v_z v_z + u_z u_z v_z v_z$$

$$- u_x v_x u_x v_x - u_y v_y u_y v_y - u_z v_z u_z v_z$$

$$- 2u_x v_x u_y v_y - 2u_y v_y u_z v_z - 2u_z v_z u_x v_x$$

$$= u_y u_y v_x v_x + u_z u_z v_x v_x + u_x u_x v_y v_y + u_z u_z v_y v_y + u_x u_x v_z v_z + u_y u_y v_z v_z$$

$$- 2u_x v_x u_y v_y - 2u_y v_y u_z v_z - 2u_z v_z u_x v_x$$





- $\vec{u} \times \vec{v} = |\vec{u}| |\vec{v}| \sin \theta \hat{n}$ 전개

$$|\vec{u} \times \vec{v}|^2 = |\vec{u}|^2 |\vec{v}|^2 \sin^2 \theta$$

$$= u_y u_y v_x v_x + u_z u_z v_x v_x + u_x u_x v_y v_y + u_z u_z v_y v_y + u_x u_x v_z v_z + u_y u_y v_z v_z \\ - 2u_x v_x u_y v_y - 2u_y v_y u_z v_z - 2u_z v_z u_x v_x$$

$$= u_y^2 v_x^2 + u_z^2 v_x^2 + u_x^2 v_y^2 + u_z^2 v_y^2 + u_x^2 v_z^2 + u_y^2 v_z^2 - 2u_x v_x u_y v_y - 2u_y v_y u_z v_z - 2u_z v_z u_x v_x$$

$$= u_y^2 v_z^2 - 2u_y v_z u_z v_y + u_z^2 v_y^2 + u_z^2 v_x^2 - 2u_z v_x u_x v_z + u_x^2 v_z^2 + u_x^2 v_y^2 - 2u_x v_y u_y v_x + u_y^2 v_x^2$$

$$= (u_y v_z - u_z v_y)^2 + (u_z v_x - u_x v_z)^2 + (u_x v_y - u_y v_x)^2$$

$$= (u_y v_z - u_z v_y, u_z v_x - u_x v_z, u_x v_y - u_y v_x)^2$$

$$= \begin{vmatrix} \hat{x} & \hat{y} & \hat{z} \\ u_x & u_y & u_z \\ v_x & v_y & v_z \end{vmatrix}^2$$





● 외적의 성질

$$\begin{aligned}\vec{v} \times \vec{u} &= \begin{vmatrix} \hat{x} & \hat{y} & \hat{z} \\ v_x & v_y & v_z \\ u_x & u_y & u_z \end{vmatrix} = (v_y u_z - v_z u_y, v_z u_x - v_x u_z, v_x u_y - u_y v_x) \\ &= -(v_z u_y - v_y u_z, v_x u_z - v_z u_x, u_y v_x - v_x u_y) \\ &= -\vec{u} \times \vec{v}\end{aligned}$$

$$\vec{u} \times \vec{v} = -\vec{v} \times \vec{u}$$

● 외적 덧셈 분배 법칙

$$\vec{A} \times (\vec{B} + \vec{C}) = \vec{A} \times \vec{B} + \vec{A} \times \vec{C}$$

● 3항 연산

$$\vec{A} \times (\vec{B} \times \vec{C}) = (\vec{A} \cdot \vec{C})\vec{B} - (\vec{A} \cdot \vec{B})\vec{C}$$

$$(\vec{A} \times \vec{B}) \times \vec{C} = -\vec{C} \times (\vec{A} \times \vec{B}) = -(\vec{C} \cdot \vec{B})\vec{A} + (\vec{C} \cdot \vec{A})\vec{B} = (\vec{A} \cdot \vec{C})\vec{B} - (\vec{B} \cdot \vec{C})\vec{A}$$

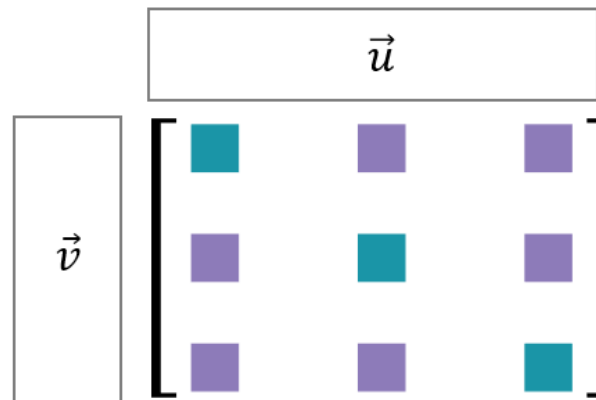
$$\vec{A} \times (\vec{B} \times \vec{C}) \neq (\vec{A} \times \vec{B}) \times \vec{C}$$





● 내적과 외적이 가지는 성질 비교

- 외적이 사용하는 요소
- 내적이 사용하는 요소



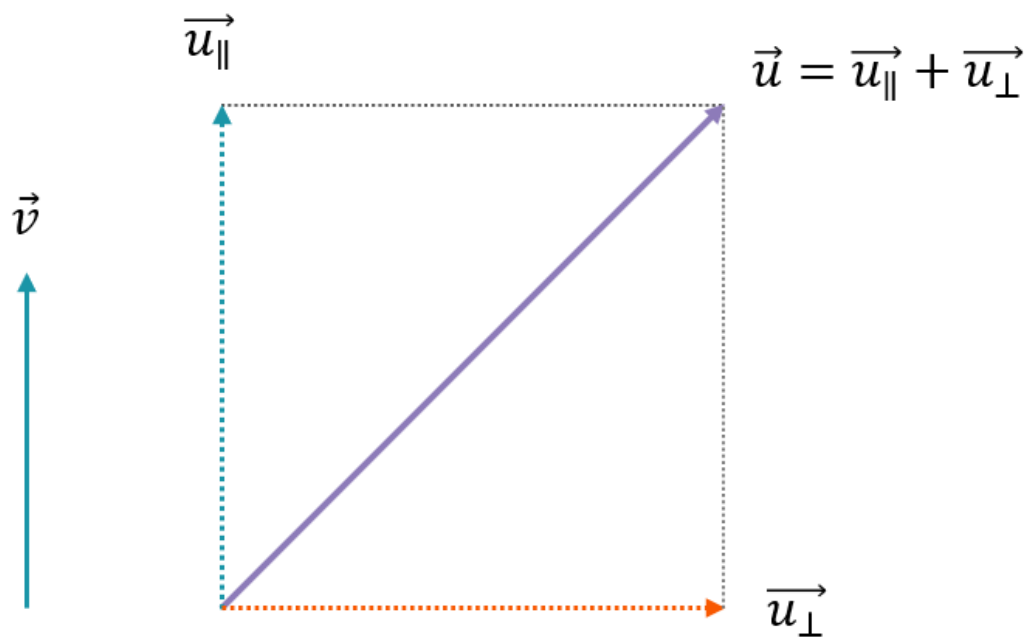
● 벡터 내적 외적 차이

	내적	외적
계산 결과	스칼라	벡터
교환 법칙	성립함	성립하지 않음
결합 법칙	성립하지 않음	성립하지 않음
분배 법칙	성립함	성립함
연산 방법	같은 위치의 요소만 사용	다른 위치의 요소만 사용



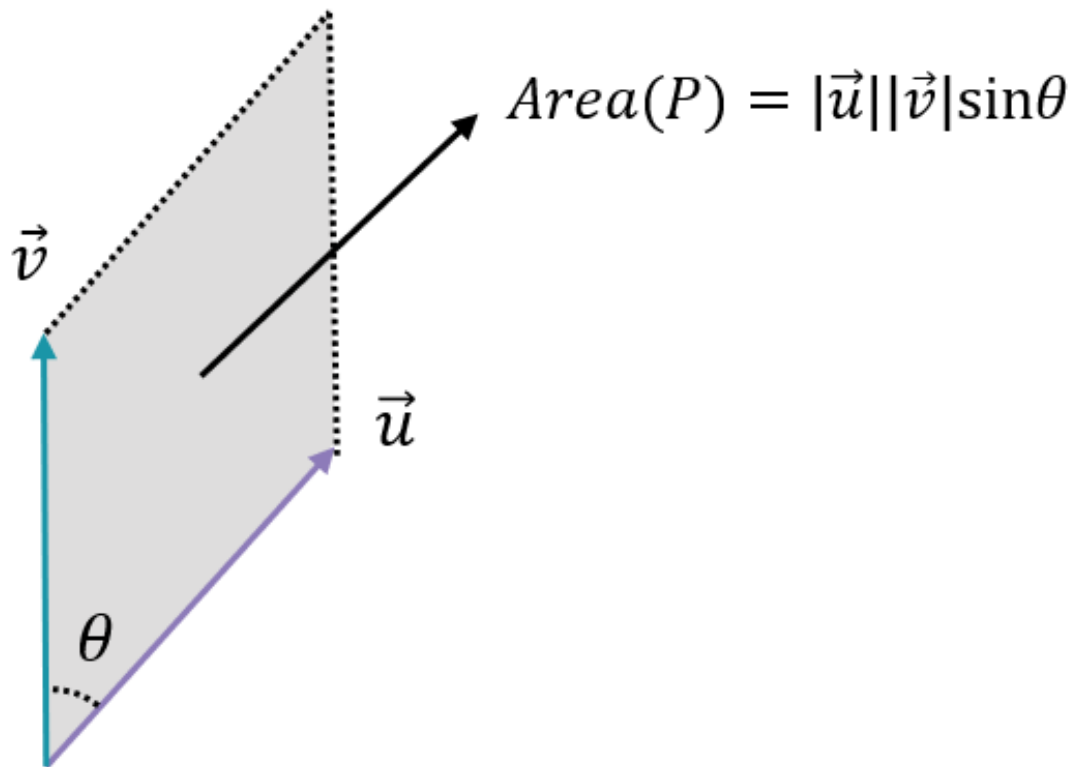


- 두 벡터의 수직: 내적 결과 0
- 두 벡터의 수평: 외적 결과 0-벡터
- 벡터를 수평 성분, 수직 성분 분리





- 두 벡터 $\vec{u}, \vec{v}$ 가 만들어내는 평행사변형의 넓이



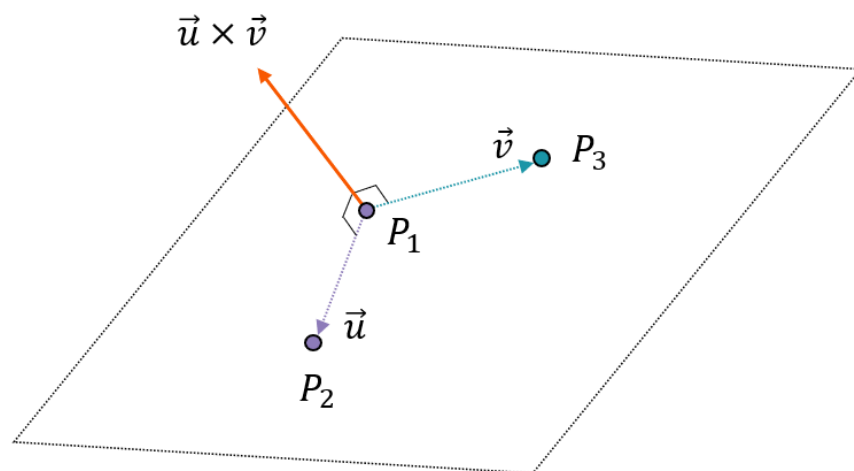


- 두 벡터의 외적 $\rightarrow$ 두 벡터에 직교

$$\vec{u} \cdot \vec{u} \times \vec{v} = \begin{vmatrix} u_x & u_y & u_z \\ u_x & u_y & u_z \\ v_x & v_y & v_z \end{vmatrix} = 0$$

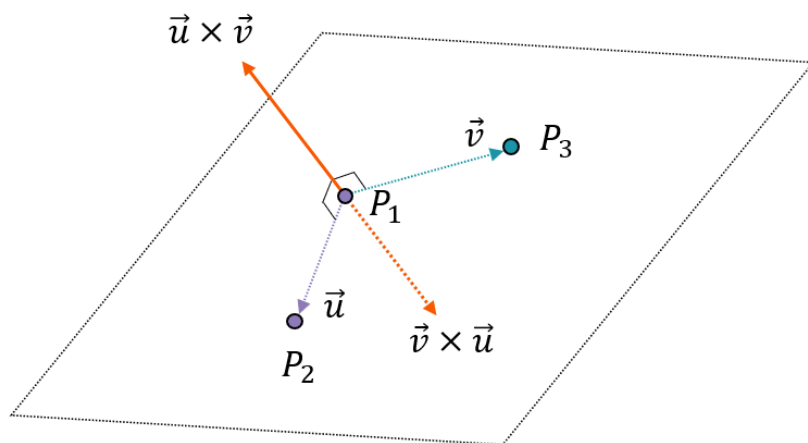
$$\vec{v} \cdot \vec{u} \times \vec{v} = \begin{vmatrix} v_x & v_y & v_z \\ u_x & u_y & u_z \\ v_x & v_y & v_z \end{vmatrix} = 0$$

- 법선 벡터를 생성하는 외적 연산





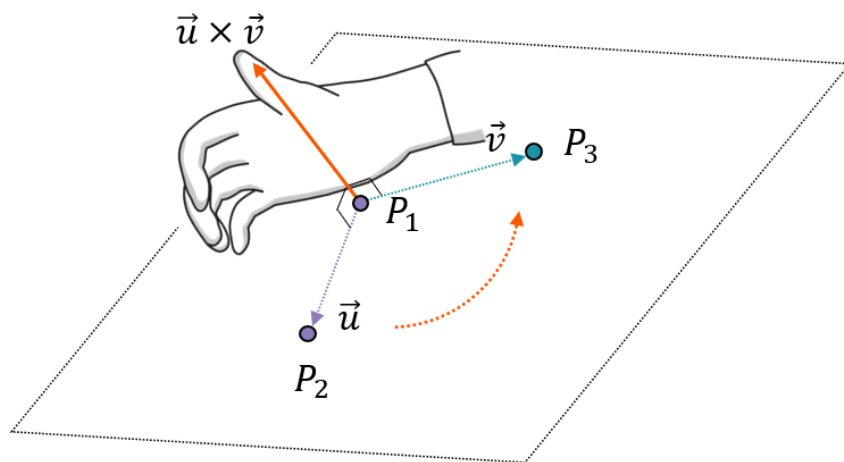
- 외적의 연산 순서에 따른 결과 비교



법선 벡터

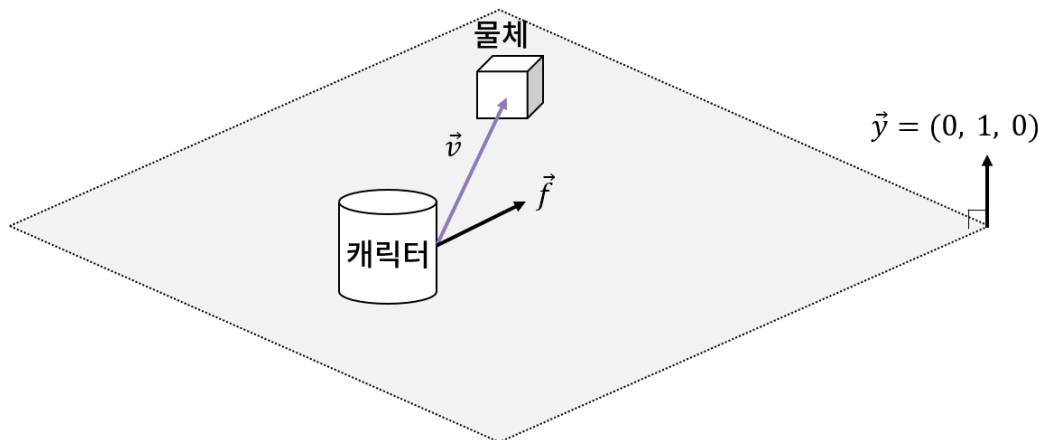
$$\frac{\vec{u} \times \vec{v}}{|\vec{u} \times \vec{v}|}$$

- 오른손 법칙으로 외적 방향 결정

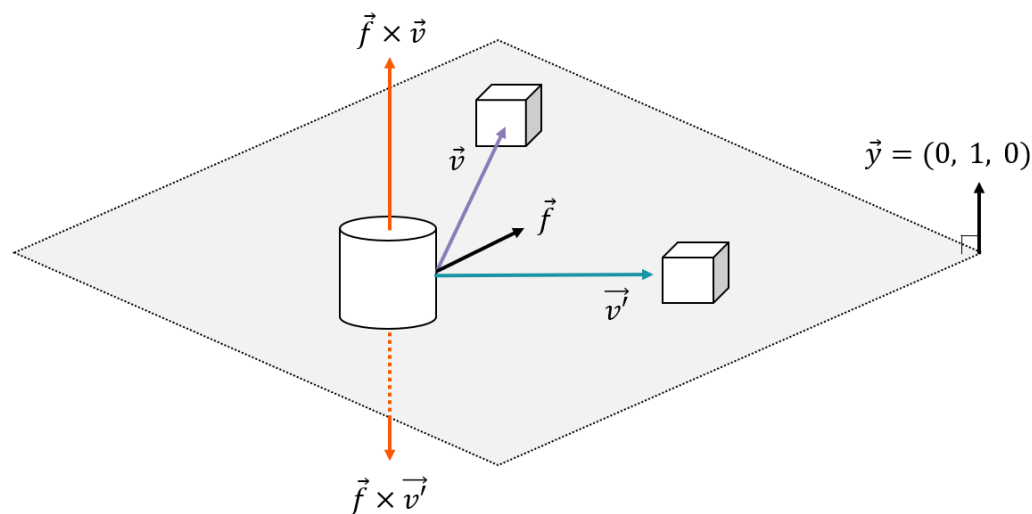




- 좌우 방향 판별을 위한 예시



- 물체 위치에 따라 달라지는 외적의 결과



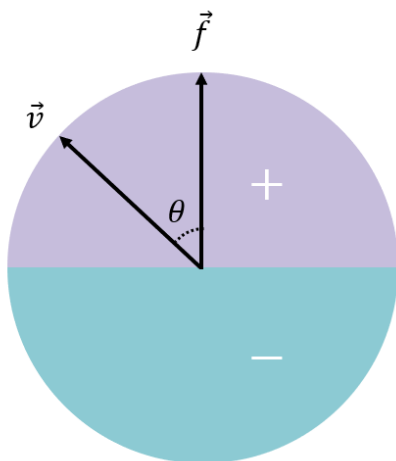


● 물체의 좌우 방향 판별을 위한 수식

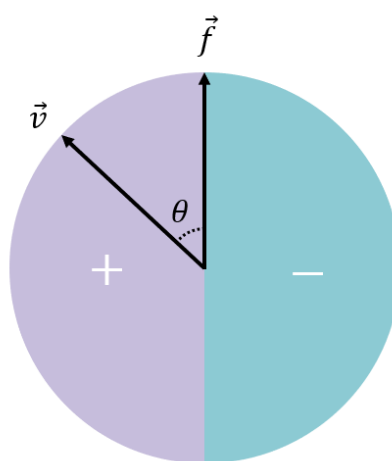
물체의 위치	판별식의 값
물체가 시선 방향에서 왼쪽에 있는 경우	$(\vec{f} \times \vec{v}) \cdot \vec{y} > 0$
물체가 시선 방향에서 오른쪽에 있는 경우	$(\vec{f} \times \vec{v}) \cdot \vec{y} < 0$
물체가 정확히 시선 방향과 일치하는 경우	$(\vec{f} \times \vec{v}) \cdot \vec{y} = 0$

● 삼각함수에 비례하는 내적과 외적이 가지는 부호 영역

(a) 내적



(b) 외적

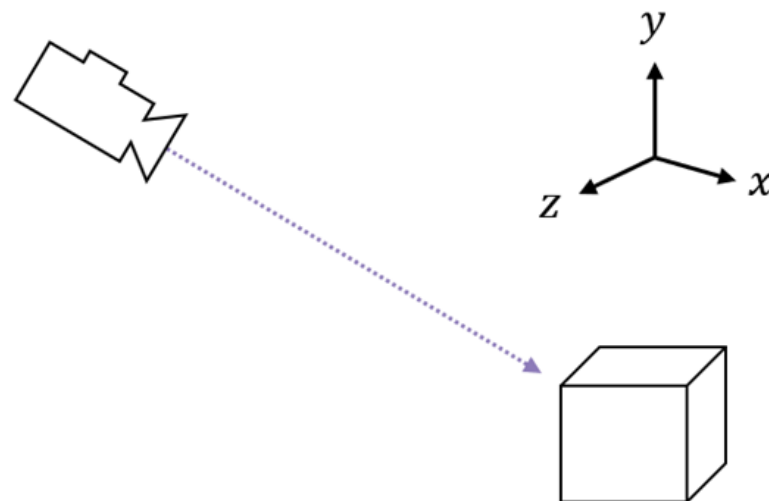


	내적	외적
방향 판별	앞, 뒤	왼쪽, 오른쪽

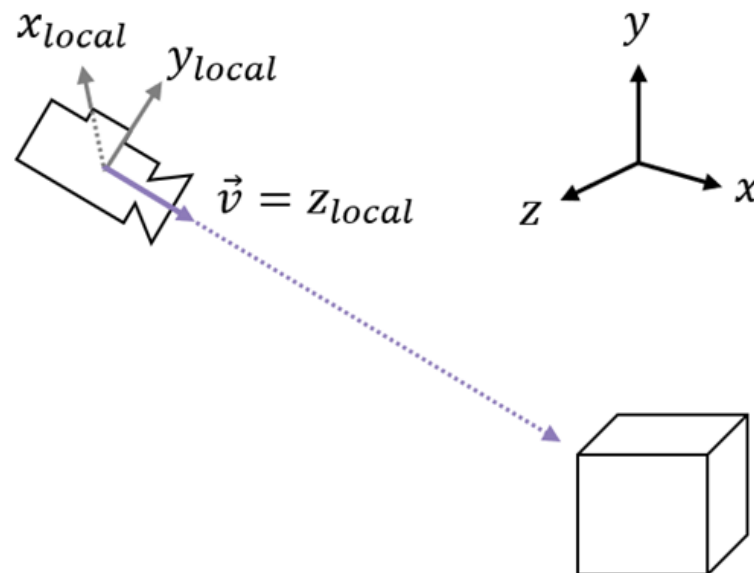




- 물체를 바라보는 카메라의 설정

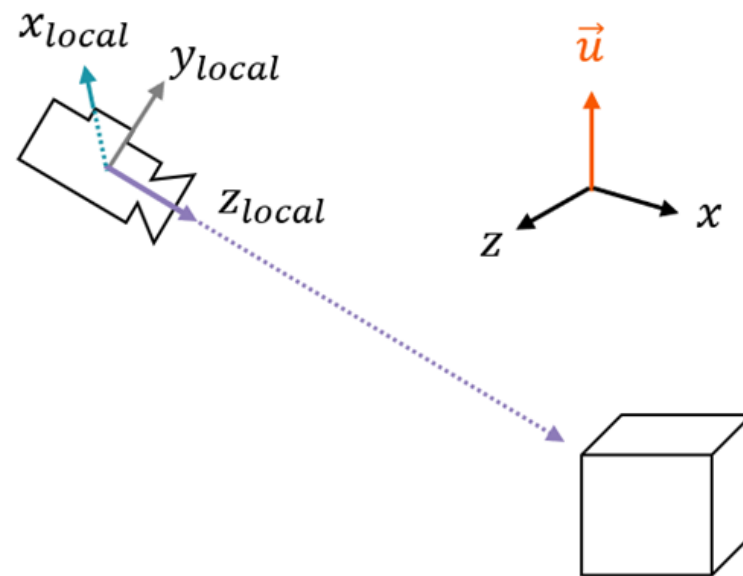


- 카메라의 로컬 z 축 구하기

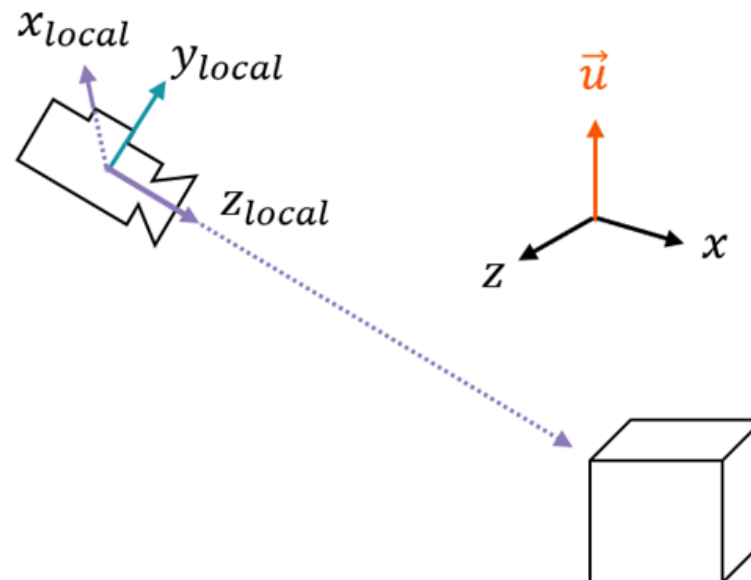




- 카메라의 로컬 x 축 구하기



- 뷰 공간을 구성하는 y 축 구하기



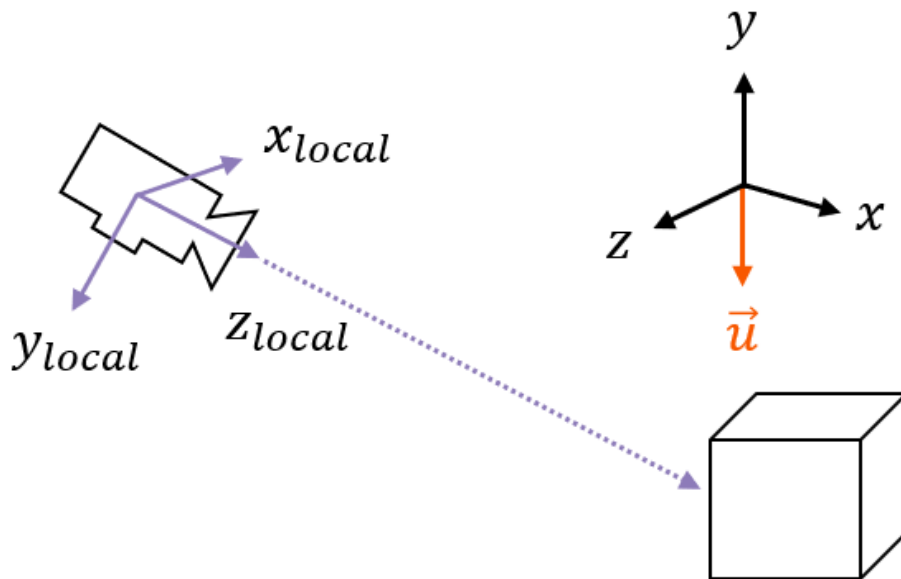


- 카메라 기저 벡터

$$\hat{x} = \frac{\vec{u} \times \vec{v}}{|\vec{u} \times \vec{v}|} \quad \hat{y} = \hat{z} \times \hat{x} \quad \hat{z} = \frac{\vec{v}}{|\vec{v}|}$$

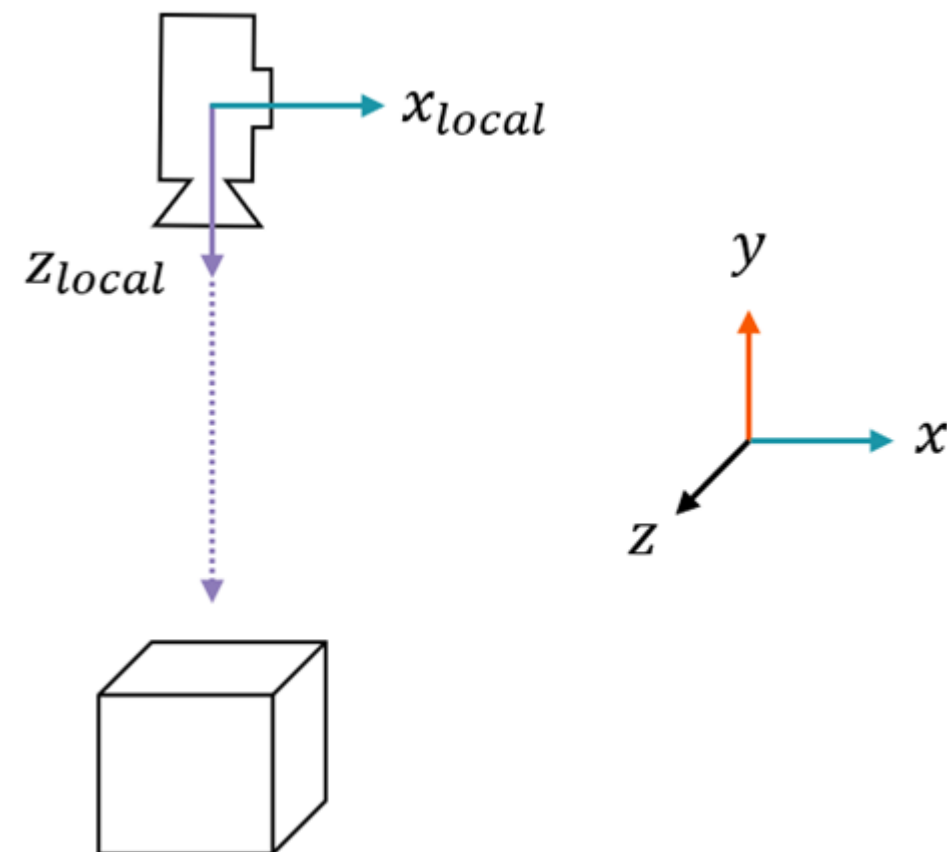
- 거꾸로 뒤집힌 카메라의 뷰 공간

$$R = \begin{bmatrix} x_x & x_y & x_z & 0 \\ y_x & y_y & y_z & 0 \\ z_x & z_y & z_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$





- 카메라의 시선 방향이 월드 공간 y 축과 평행한 경우
 - ◆ 외적 결과 $\rightarrow 0$ -벡터
 - ◆ 외적 계산 대신 로컬 z 축에 직교하는 로컬 x 축의 값 수동 지정



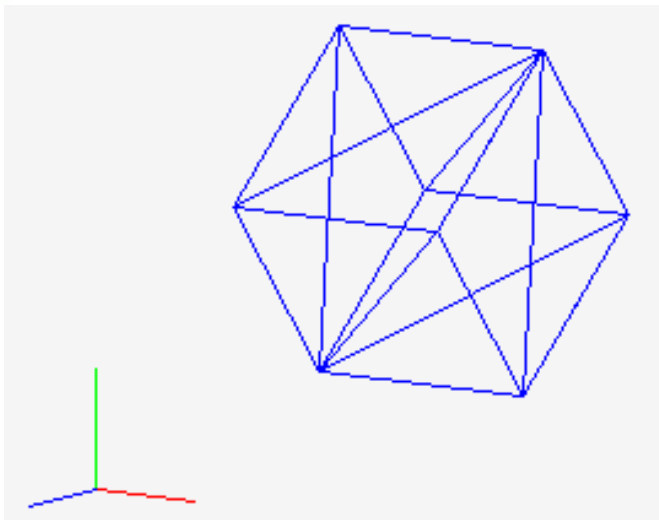
```
// 단위 z축 생성
localZ = (InTargetPosition
          - _Transform.GetPosition()).GetNormalize();
// 시선 방향과 월드 Y축이 평행한 경우
if (Math::Abs(localZ.Y) >= (1.f - SMALL_NUMBER))
{
    // 특이 상황에서 로컬 x 좌표 값을 임의로 지정.
    localX = Vector3::UnitX;
}
else
{
    localX = InUp.Cross(localZ).GetNormalize();
}
```





실습 11-1 항상 물체를 바라보는 카메라의 구현

Heesung Oh
Copyright All rights reserved



```
void CameraObject::SetLookAtRotation(const Vector3& InTargetPosition, const Vector3& InUp) {  
    Vector3 localX, localY, localZ;  
    // 단위 Z축 생성  
    localZ = (InTargetPosition - _Transform.GetPosition()).GetNormalize();  
    // 시선 방향과 월드 Y축이 평행한 경우  
    if (Math::Abs(localZ.Y) >= (1.f - SMALL_NUMBER)) {  
        // 특이 상황에서 로컬 X 좌표 값을 임의로 지정.  
        localX = Vector3::UnitX;  
    } else {  
        localX = InUp.Cross(localZ).GetNormalize();  
    }  
  
    // 두 벡터가 직교하고 크기가 1이면 결과는 1을 보장  
    localY = localZ.Cross(localX);  
    _Transform.SetLocalAxes(localX, localY, localZ);  
}
```





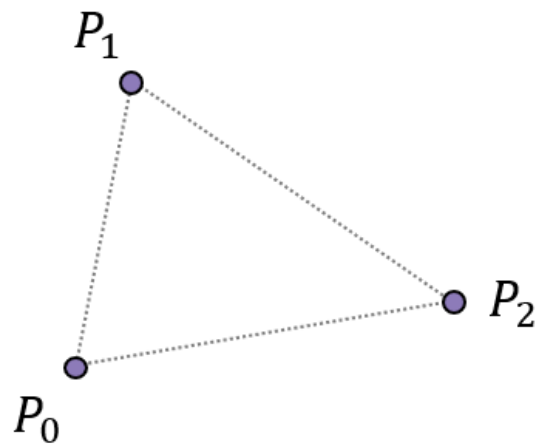
● 정점 버퍼와 인덱스 버퍼

정점 버퍼

인덱스
0
1
2

인덱스 버퍼

인덱스	정점 인덱스
0	0
1	1
2	2





● 점의 순서에 따른 면의 방향

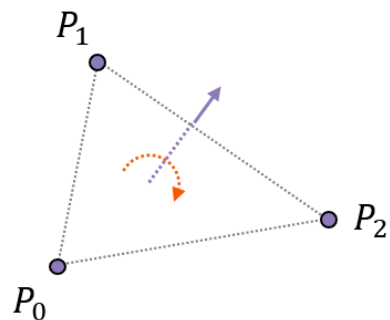
(a)

정점 버퍼

인덱스
0
1
2

인덱스 버퍼

인덱스	정점 인덱스
0	0
1	1
2	2



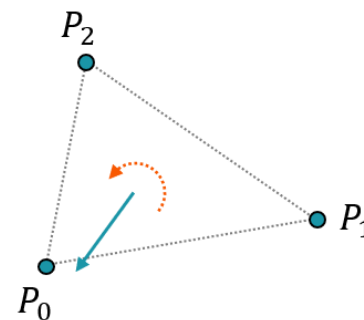
(b)

정점 버퍼

인덱스
0
1
2

인덱스 버퍼

인덱스	정점 인덱스
0	0
1	2
2	1

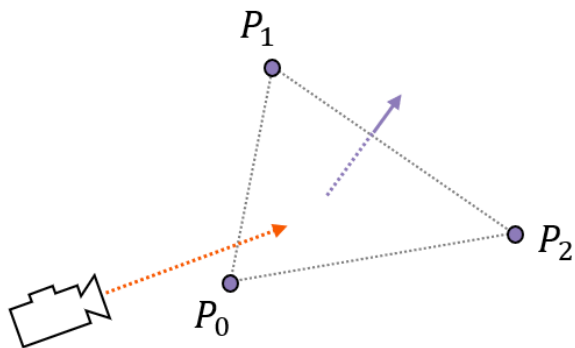




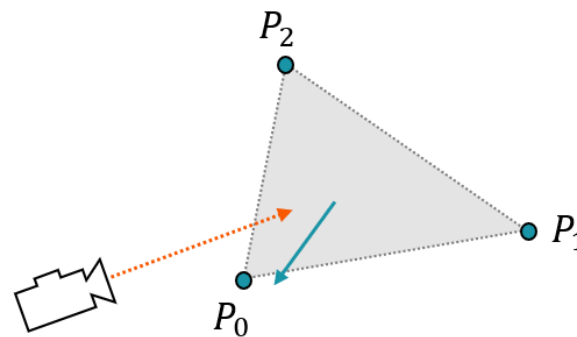
- 백페이스 컬링 원리

- ◆ 카메라의 시선 방향, 삼각형의 법선 내적

(a)



(b)

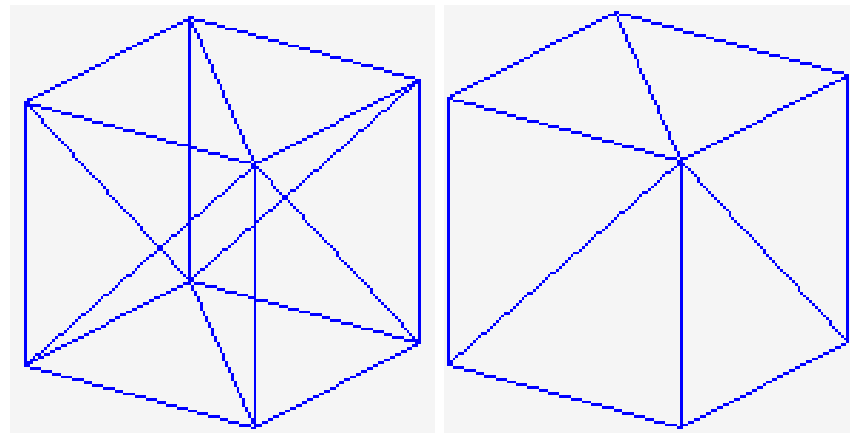




- 세 점 P_0, P_1, P_2

$$\vec{n} = \overrightarrow{P_0P_1} \times \overrightarrow{P_0P_2}$$

$$(n_x, n_y, n_z) \cdot (0, 0, -1) = -n_z$$



```
auto& r = GetRenderer();  
const GameEngine& g = Get3DGameEngine();  
  
if (useBackfaceCulling) {  
    // 백페이스 컬링 ( 뒷면이면 그리기 생략 )  
    Vector3 edge1 = (InVertices[1].Position - InVertices[0].Position).ToVector3();  
    Vector3 edge2 = (InVertices[2].Position - InVertices[0].Position).ToVector3();  
    Vector3 faceNormal = edge1.Cross(edge2);  
    Vector3 viewDirection = -Vector3::UnitZ;  
    if (faceNormal.Dot(viewDirection) >= 0.f) {  
        return;  
    }  
}
```



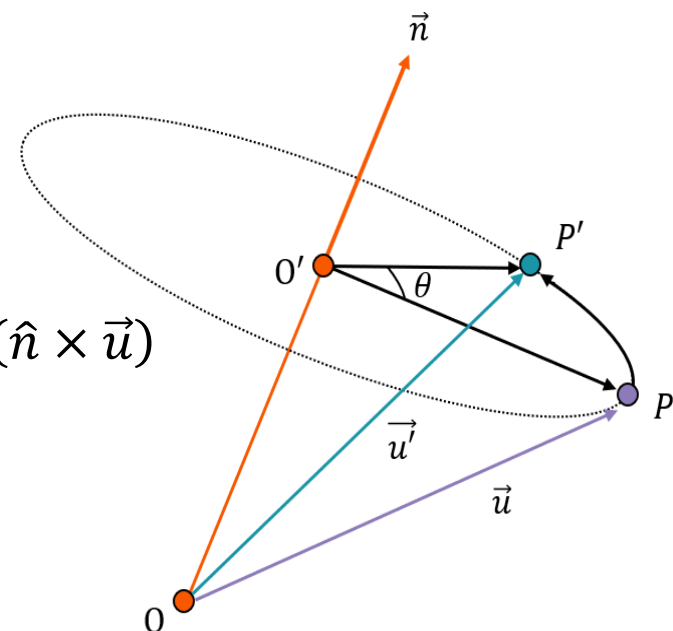
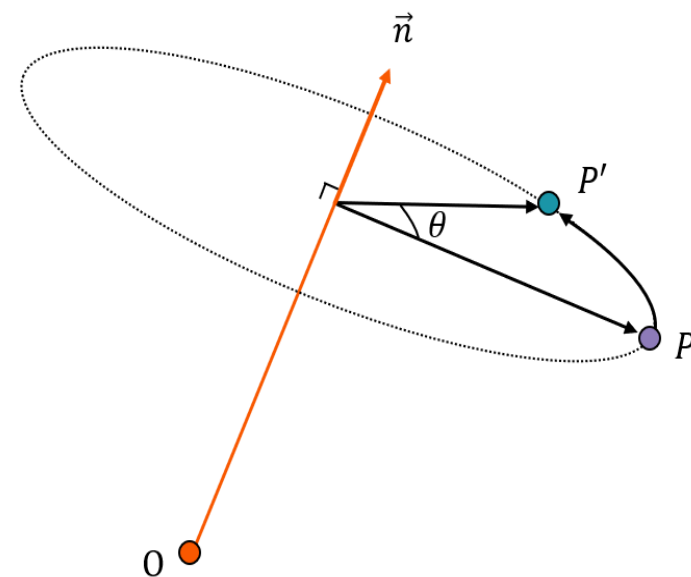


● 임의의 축에 대한 회전의 개요

● 회전 평면의 설정

$$P = (x, y, z, 1) \quad \vec{u} = P - 0 = (x, y, z, 0)$$

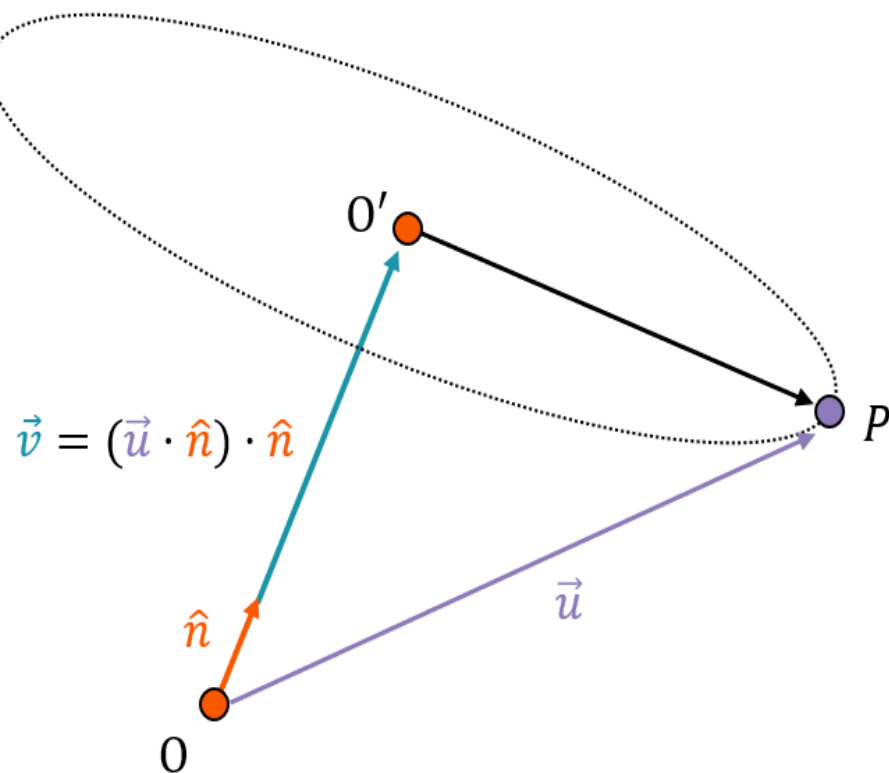
$$\vec{u} = \cos \theta \cdot \vec{u} + (1 - \cos \theta) \cdot (\vec{u} \cdot \hat{n}) \cdot \hat{n} + \sin \theta \cdot (\hat{n} \times \vec{u})$$





- 벡터 내적의 투영 공식으로 구한 벡터 $\vec{v}$

$$\overrightarrow{00'} = \vec{v} = (\vec{u} \cdot \hat{n}) \cdot \hat{n}$$



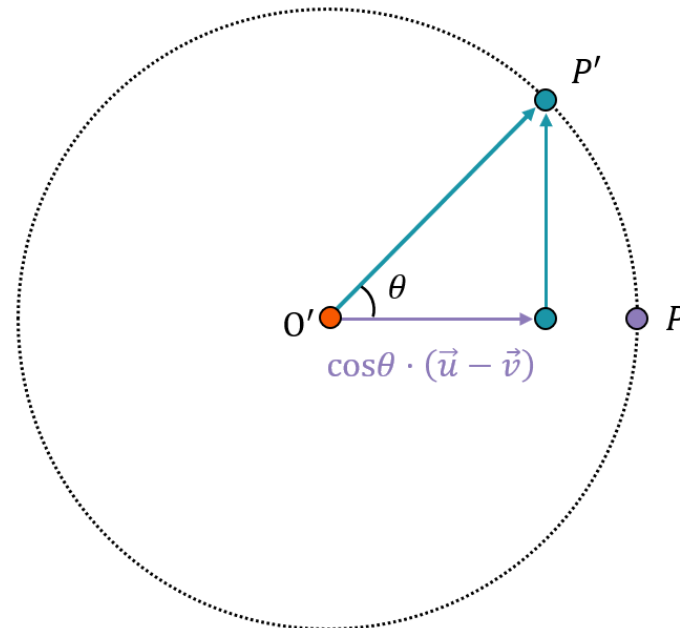
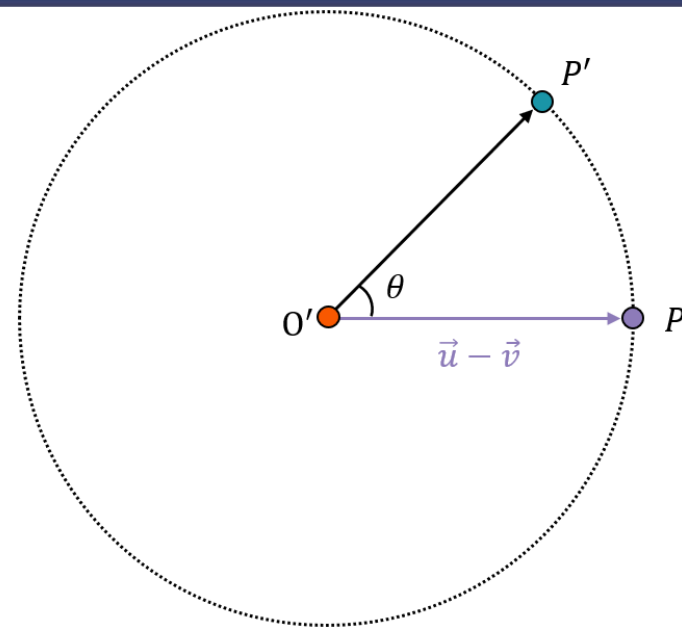


- 회전 평면에서의 회전

$$\overrightarrow{O'P} = \vec{u} - \vec{v}$$

- 회전한 점 P' 를 구하기 위한
가로 벡터의 계산

$$\overrightarrow{O'P'} = \cos \theta \cdot \overrightarrow{O'P} + \sin \theta \cdot \overrightarrow{O'Q}$$





- 회전 평면에서 벡터 $\overrightarrow{O'P}$ 에 직교하는 벡터 $\overrightarrow{O'Q}$

$$\overrightarrow{O'P'} = \cos \theta \cdot \overrightarrow{O'P} + \sin \theta \cdot \overrightarrow{O'Q}$$

$$\vec{v} = (\vec{u} \cdot \hat{n}) \cdot \hat{n}$$

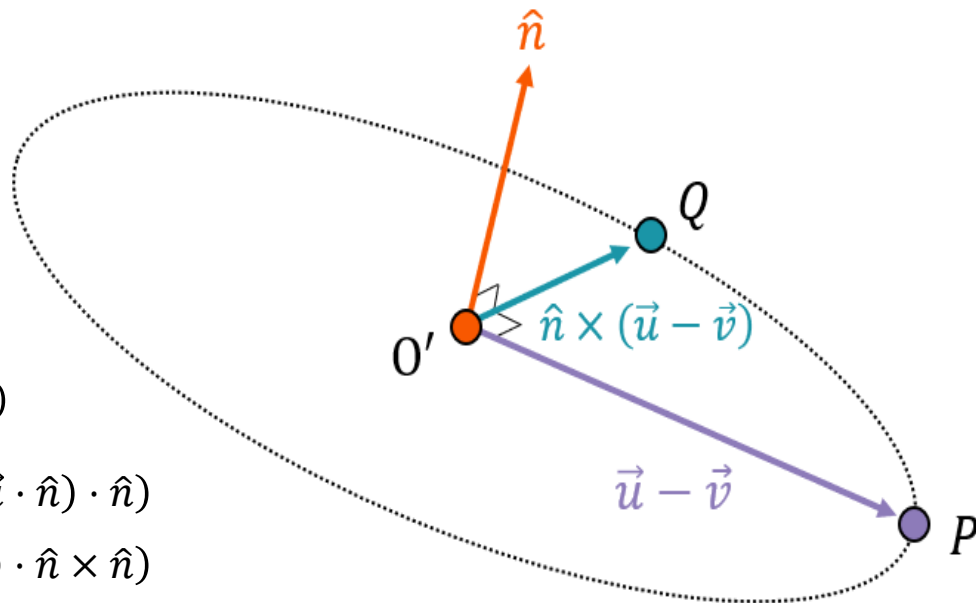
$$\overrightarrow{O'P'} = \cos \theta \cdot (\vec{u} - \vec{v}) + \sin \theta \cdot (\hat{n} \times (\vec{u} - \vec{v}))$$

$$= \cos \theta \cdot (\vec{u} - \vec{v}) + \sin \theta \cdot (\hat{n} \times \vec{u} - \hat{n} \times \vec{v})$$

$$= \cos \theta \cdot (\vec{u} - \vec{v}) + \sin \theta \cdot (\hat{n} \times \vec{u} - \hat{n} \times (\vec{u} \cdot \hat{n}) \cdot \hat{n})$$

$$= \cos \theta \cdot (\vec{u} - \vec{v}) + \sin \theta \cdot (\hat{n} \times \vec{u} - (\vec{u} \cdot \hat{n}) \cdot \hat{n} \times \hat{n})$$

$$= \cos \theta \cdot (\vec{u} - \vec{v}) + \sin \theta \cdot (\hat{n} \times \vec{u})$$





● 최종 벡터 $\overrightarrow{O'P}$ 의 계산

$$\vec{v} = (\vec{u} \cdot \hat{n}) \cdot \hat{n}$$

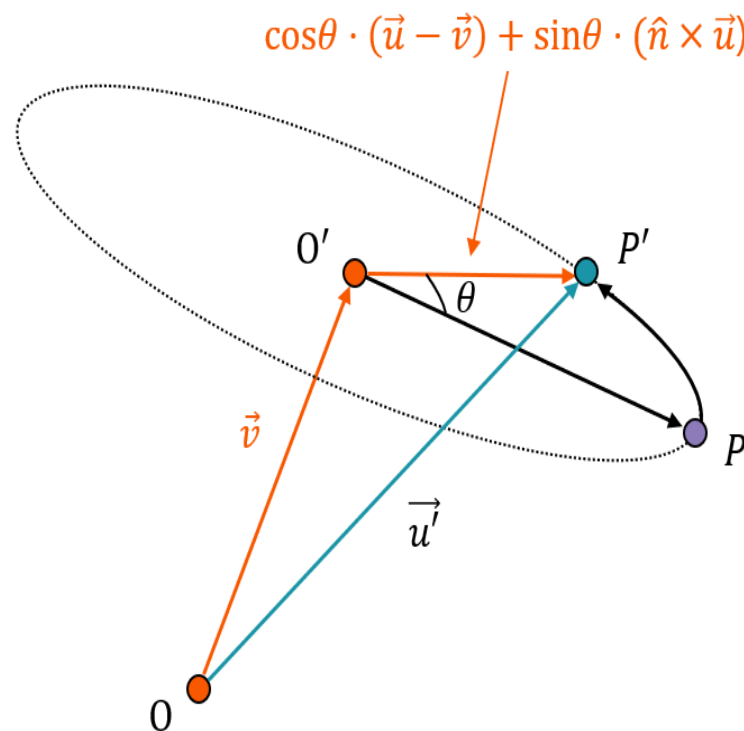
$$\overrightarrow{OP'} = \vec{v} + \overrightarrow{OP'}$$

$$\overrightarrow{O'P'} = \cos \theta \cdot (\vec{u} - \vec{v}) + \sin \theta \cdot (\hat{n} \times \vec{u})$$

$$\overrightarrow{OP'} = \vec{v} + \cos \theta \cdot (\vec{u} - \vec{v}) + \sin \theta \cdot (\hat{n} \times \vec{u})$$

$$= \cos \theta \cdot \vec{u} + (1 - \cos \theta) \cdot \vec{v} + \sin \theta \cdot (\hat{n} \times \vec{u})$$

$$= \cos \theta \cdot \vec{u} + (1 - \cos \theta)(\vec{u} \cdot \hat{n}) \cdot \hat{n} + \sin \theta \cdot (\hat{n} \times \vec{u})$$

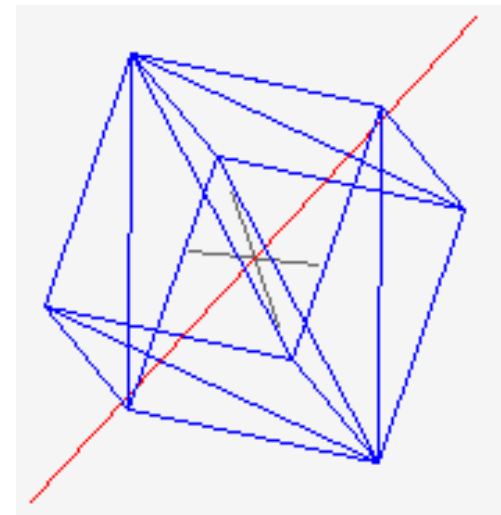


● 로드리게스 회전 공식

$$\vec{u}' = \cos \theta \cdot \vec{u} + (1 - \cos \theta)(\vec{u} \cdot \hat{n}) \cdot \hat{n} + \sin \theta \cdot (\hat{n} \times \vec{u})$$



- 로드리게스 회전 공식은
sin, cos 계산 필요
→ 행렬 변환이 용이한 사원수 사용



```
float sin = 0.f, cos = 0.f;  
Math::GetSinCos(sin, cos, thetaDegree);  
  
Vector3 u = v.Position.ToVector3();  
  
float udotn = u.Dot(n);  
  
Vector3 ncrossu = n.Cross(u);  
  
Vector3 result  
= Vector3(u * cos + n * ((1.f - cos) * udotn) + ncrossu * sin) * InScale;  
  
v.Position = InMatrix * Vector4(result);
```





- 벡터 3중 곱

$$\vec{u} \cdot (\vec{v} \cdot \vec{w})$$

$$\vec{u} \cdot (\vec{v} \times \vec{w})$$

$$\vec{u} \times (\vec{v} \cdot \vec{w})$$

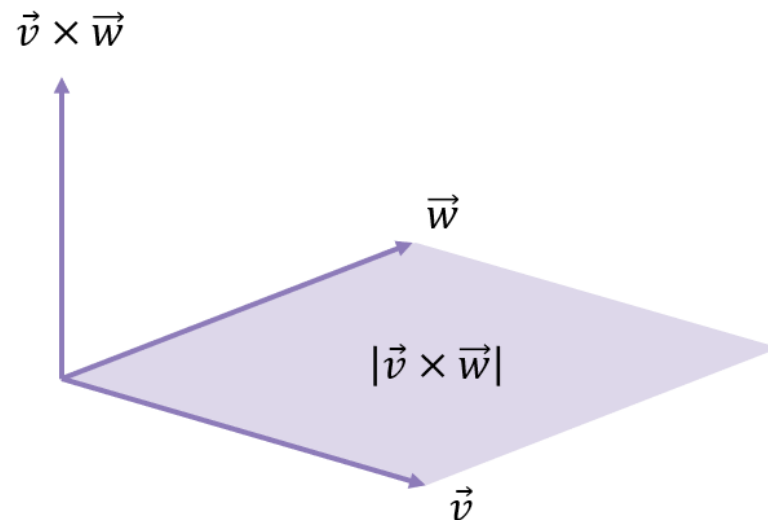
$$\vec{u} \times (\vec{v} \times \vec{w})$$





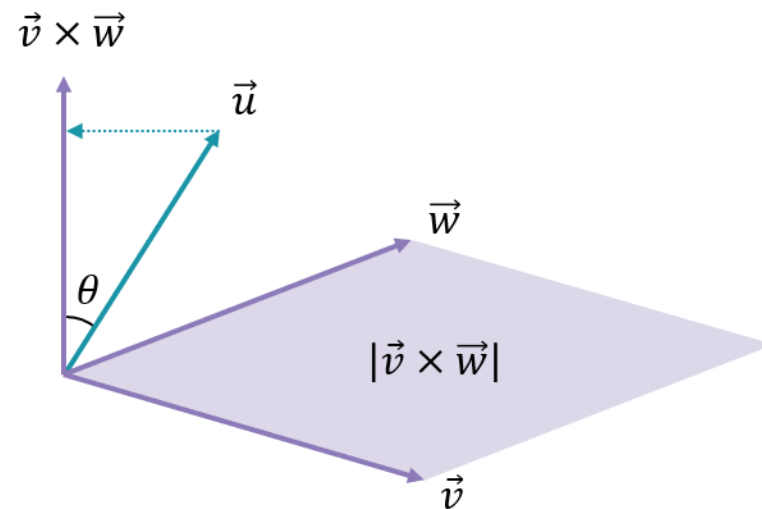
- 외적의 크기

$$(\vec{v} \times \vec{w})$$



- 법선 벡터에 투영한 벡터의 크기

$$(\vec{v} \times \vec{w}), \vec{u}$$

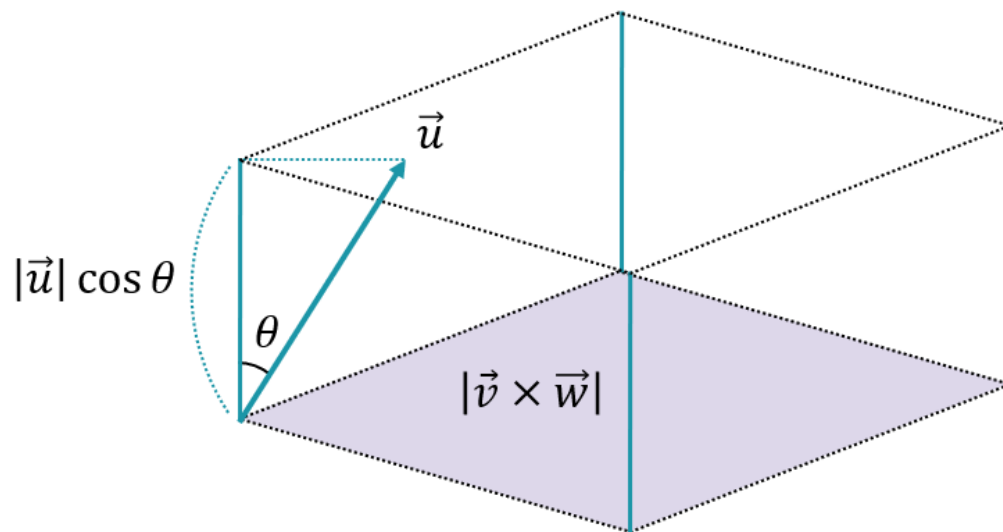




- 육면체의 부피

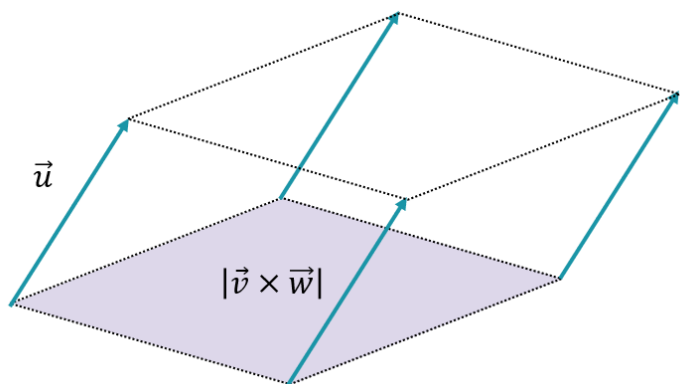
$$|\vec{u}| \cos \theta$$

$$|\vec{v} \times \vec{w}|$$

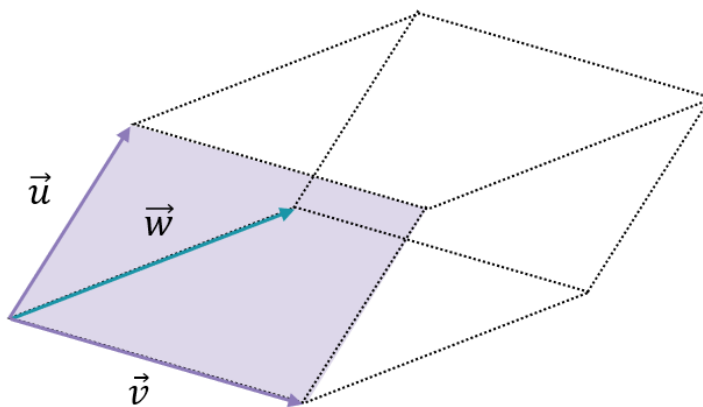
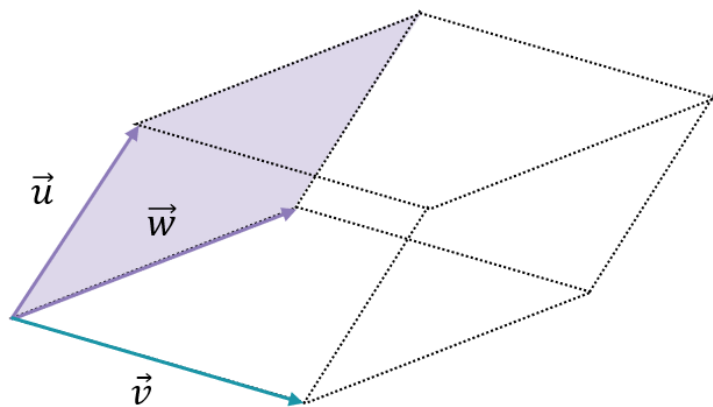




- 세 벡터로 구성된 평행육면체



- 순서를 바꿔도 결과는 동일한 스칼라 삼중곱 예시





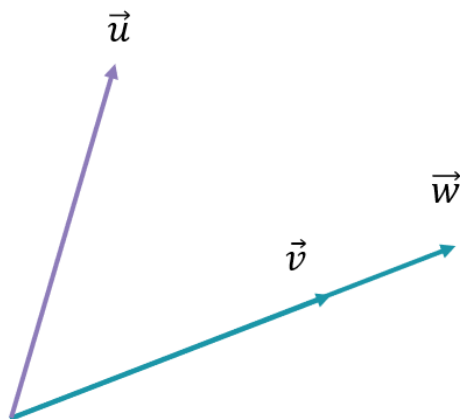
- 순서를 바꿔도 동일한 결과

$$\vec{u} \cdot (\vec{v} \times \vec{w}) \quad \vec{v} \cdot (\vec{w} \times \vec{u}) \quad \vec{w} \cdot (\vec{u} \times \vec{v})$$

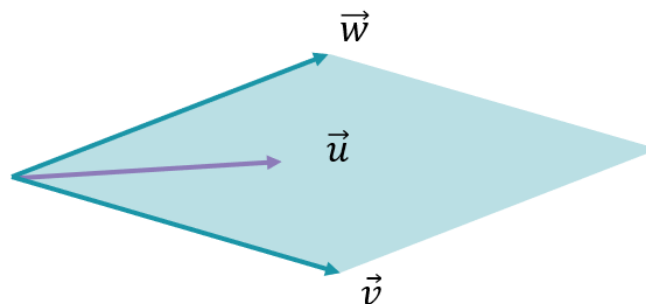
- 세 벡터가 모두 선형 독립이 아닌 경우

- 삼중곱 $\rightarrow 0$

(a)



(b)





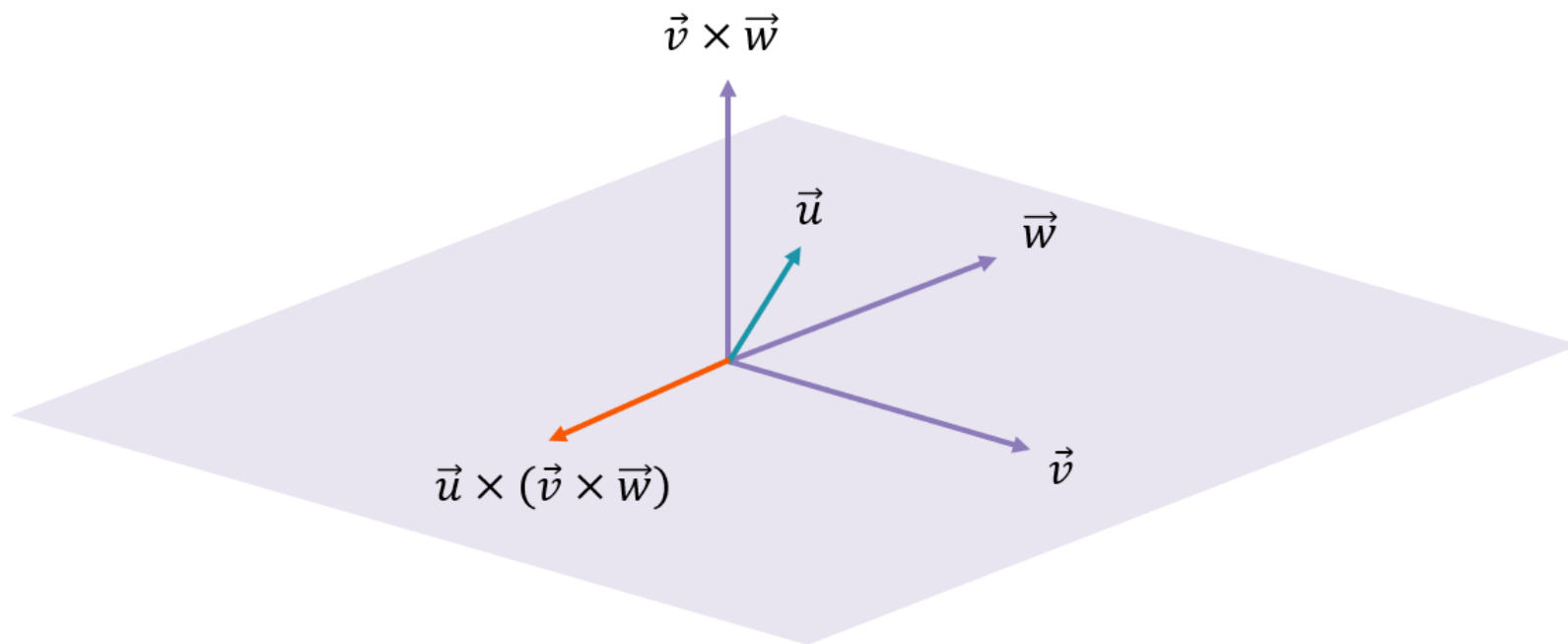
- 벡터 삼중곱 라그랑주 공식(Lagrange's formula)

$$\begin{aligned}(\vec{u} \times (\vec{v} \times \vec{w}))_z &= u_x(\vec{v} \times \vec{w})_y - u_y(\vec{v} \times \vec{w})_x \\&= u_x(v_z w_x - v_x w_z) - u_y(v_y w_z - v_z w_y) \\&= u_x v_z w_x - u_x v_x w_z - u_y v_y w_z + u_y v_z w_y \\&= u_x w_x v_z + u_y w_y v_z - u_x v_x w_z - u_y v_y w_z \\&= u_x w_x v_z + u_y w_y v_z + u_z w_z v_z - u_z w_z v_z - u_x v_x w_z - u_y v_y w_z \\&= (u_x w_x + u_y w_y + u_z w_z) v_z - (u_z v_z + u_x v_x + u_y v_y) w_z \\&= (\vec{u} \cdot \vec{w}) v_z - (u_x v_x + u_y v_y + u_z v_z) w_z \\&= (\vec{u} \cdot \vec{w}) v_z - (\vec{u} \cdot \vec{v}) w_z \\ \vec{u} \times (\vec{v} \times \vec{w}) &= (\vec{u} \cdot \vec{w}) \vec{v} - (\vec{u} \cdot \vec{v}) \vec{w}\end{aligned}$$





- 벡터 삼중곱으로 만들어진 벡터의 결과
→ $\vec{v}$ 와 $\vec{w}$ 가 만드는 평면의 벡터

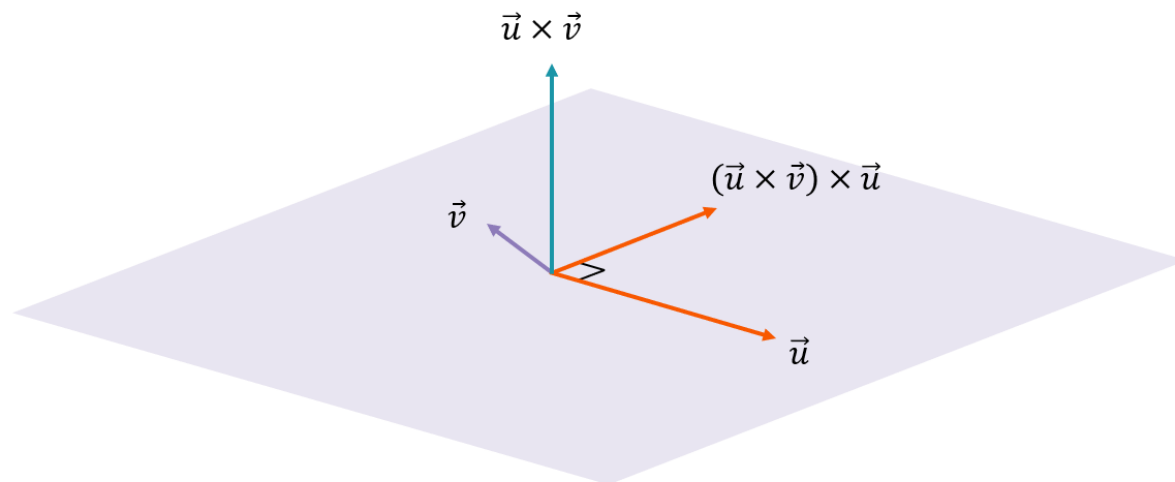




- 2차원 공간 응용

- ◆ 벡터 $\vec{u}$ 와 $\vec{v}$ 가 주어질 때 $\rightarrow$ u, v 평면에서 $\vec{u}$ 와 직교 벡터(예각)

$$(\vec{u} \times \vec{v}) \times \vec{u}$$





- 외적: 3차원 공간만 사용 가능
- 법선 벡터
- 로드리게스 회전 공식
- 라그랑주 공식
- 삼중곱 연산

